

图像变形实验报告

学号：PB23000270

姓名：李佩优

2026 年 3 月 15 日

目录

1	算法原理	2
1.1	反距离加权插值 (IDW)	2
1.2	径向基函数变换 (RBF)	2
2	实现细节	3
2.1	面向对象设计	3
2.2	IDW 的实现	3
2.3	RBF 的实现	4
2.4	图像变形流程	4
3	实验结果与分析	4
3.1	测试图像与控制点	4
3.2	IDW 变形效果	5
3.3	RBF 变形效果	5
3.4	正向映射产生的空洞问题	5
4	讨论与改进	5
5	结论	7

1 算法原理

图像变形 (Image Warping) 是指根据一组给定的控制点对 (源点与目标点) 构造一个连续映射 $\mathbf{T} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, 使得源点被映射到对应的目标点, 并平滑地扭曲整个图像。本实验实现了两种经典的插值方法: 反距离加权插值 (IDW) 和径向基函数变换 (RBF)。

1.1 反距离加权插值 (IDW)

IDW 方法由 Shepard 提出, 其核心思想是: 任意点的位移由所有控制点的位移加权平均得到, 权重与到该控制点的距离成反比。

设给定 n 个控制点 $\mathbf{p}_i$ 及其目标位置 $\mathbf{q}_i$ 。对于平面上任意点 $\mathbf{x}$, 其映射后的位置为:

$$\mathbf{T}(\mathbf{x}) = \frac{\sum_{i=1}^n w_i(\mathbf{x}) \mathbf{T}_i(\mathbf{x})}{\sum_{j=1}^n w_j(\mathbf{x})}$$

其中 $w_i(\mathbf{x}) = \|\mathbf{x} - \mathbf{p}_i\|^{-\mu}$ (通常取 $\mu = 2$), $\mathbf{T}_i(\mathbf{x})$ 是一个局部近似变换。在经典 IDW 中常取 $\mathbf{T}_i(\mathbf{x}) = \mathbf{q}_i + \mathbf{A}_i(\mathbf{x} - \mathbf{p}_i)$, 即允许每个控制点附近有一个仿射变换。矩阵 $\mathbf{A}_i$ 可通过最小化邻近控制点的误差预先计算。

另一种简化形式是只考虑平移 ($\mathbf{T}_i(\mathbf{x}) = \mathbf{q}_i$), 但这样会导致非均匀伸缩。本实验采用了带局部仿射的 IDW 以提高光滑性。

1.2 径向基函数变换 (RBF)

RBF 方法将映射表示为径向基函数的线性组合加上一个仿射项:

$$\mathbf{T}(\mathbf{x}) = \sum_{i=1}^n \alpha_i \phi(\|\mathbf{x} - \mathbf{p}_i\|) + \mathbf{A}\mathbf{x} + \mathbf{b}$$

其中 $\phi(r)$ 是径向基函数, 常见的选择有:

- 薄板样条: $\phi(r) = r^2 \ln r$
- 多二次曲面: $\phi(r) = \sqrt{r^2 + c^2}$ (本实验采用)
- 高斯函数: $\phi(r) = e^{-r^2/\sigma^2}$

系数 $\alpha_i \in \mathbb{R}^2$, 仿射矩阵 $\mathbf{A} \in \mathbb{R}^{2 \times 2}$ 和偏移向量 $\mathbf{b} \in \mathbb{R}^2$ 通过求解线性系统确定, 满足插值条件 $\mathbf{T}(\mathbf{p}_i) = \mathbf{q}_i$ 以及仿射项的附加约束 $\sum_i \alpha_i = \mathbf{0}$, $\sum_i \alpha_i \mathbf{p}_i^\top = \mathbf{0}$ 。

该线性系统可写为:

$$\begin{bmatrix} \mathbf{K} & \mathbf{P} \\ \mathbf{P}^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ \boldsymbol{\beta} \end{bmatrix} = \begin{bmatrix} \mathbf{Q} \\ \mathbf{0} \end{bmatrix}$$

其中 $K_{ij} = \phi(\|\mathbf{p}_i - \mathbf{p}_j\|)$, $\mathbf{P}$ 是 $n \times 3$ 矩阵, 第 i 行为 $[p_{ix}, p_{iy}, 1]$, $\boldsymbol{\beta}$ 包含了 $\mathbf{A}$ 和 $\mathbf{b}$ 的参数。

求解后即可得到完整的映射函数。

2 实现细节

2.1 面向对象设计

为实现算法与图像处理的解耦，设计了一个抽象基类 `Warper`，定义核心接口：

- `virtual Point warping(float x, float y) = 0;`
给定输入坐标，返回映射后的坐标。
- `virtual void preprocessing() = 0;`
根据控制点对计算插值所需的所有系数。

`IDWWarper` 和 `RBFWarper` 分别继承该类并实现各自算法。控制点通过 `AddVectors(Point start, Point end)` 添加，图像尺寸通过 `SetSize` 传入用于边界检查。

2.2 IDW 的实现

数据结构：

- `vector<Vector> vector_field_` 存储控制点对。
- `vector<Matrix> T` 存储每个控制点对应的局部线性变换矩阵 $\mathbf{A}_i$ 。

预处理：

1. 若只有一对控制点，设 $\mathbf{T}_0 = \mathbf{I}$ （单位阵）。
2. 对于每个控制点 i ，使用其他控制点 j 的位移来估计局部仿射变换。通过最小化加权误差：

$$\mathbf{A}_i = \left(\sum_{j \neq i} w_{ij} (\mathbf{p}_j - \mathbf{p}_i)(\mathbf{p}_j - \mathbf{p}_i)^\top \right)^{-1} \left(\sum_{j \neq i} w_{ij} (\mathbf{p}_j - \mathbf{p}_i)(\mathbf{q}_j - \mathbf{q}_i)^\top \right)^\top$$

其中 $w_{ij} = 1/\|\mathbf{p}_j - \mathbf{p}_i\|^2$ 。代码中通过矩阵运算实现了该最小二乘解。

变形映射：对于点 $\mathbf{x}$ ：

1. 若与某个控制点距离小于阈值（`SNAP_DIST_SQ`），直接返回其目标点。
2. 计算权重 $w_i = 1/\|\mathbf{x} - \mathbf{p}_i\|^2$ 及总和 s 。
3. 加权平均：

$$\mathbf{T}(\mathbf{x}) = \frac{1}{s} \sum_i w_i [\mathbf{q}_i + \mathbf{A}_i(\mathbf{x} - \mathbf{p}_i)]$$

2.3 RBF 的实现

依赖：使用 Eigen 库进行矩阵运算。

预处理：

1. 构建矩阵 $\mathbf{P}$ ($n \times 3$) 和 $\mathbf{Q}$ ($n \times 2$)。
2. 求解最小二乘问题得到初始仿射变换 $\mathbf{A}$ 和 $\mathbf{b}$ ：
 $\mathbf{X} = \mathbf{P_aug.colPivHouseholderQr()}.solve(\mathbf{Q});$
其中 $\mathbf{P_aug} = [\mathbf{P}, \mathbf{1}]$ 。

3. 计算残差 $\mathbf{R} = \mathbf{Q} - (\mathbf{A}\mathbf{P}^T + \mathbf{b})$ 。

4. 构建径向基矩阵 $\mathbf{G}$ ：

$$G_{ij} = \sqrt{\|\mathbf{p}_i - \mathbf{p}_j\|^2 + r_i r_j}, \text{ 其中 } r_i \text{ 取为到最近邻控制点的距离。}$$

5. 求解 $\mathbf{G}\boldsymbol{\alpha} = \mathbf{R}$ 得到系数矩阵 $\boldsymbol{\alpha}$ ($n \times 2$)。

变形映射：对于点 $\mathbf{x}$ ：

1. 计算仿射部分： $\text{source_pt} = \mathbf{A} * \text{target_pt} + \mathbf{b}$ 。

2. 累加径向基部分：

$$\text{source_pt} += \sum_i \alpha_i \sqrt{\|\mathbf{x} - \mathbf{p}_i\|^2 + r_i^2}$$

3. 返回映射后的坐标。

2.4 图像变形流程

图像变形在 `WarpingWidget::warping()` 中执行：

1. 根据用户选择的变形类型创建对应的 Warper 对象。
2. 传入所有控制点对，调用 `preprocessing()`。
3. 遍历原图每个像素 (x, y) ，调用 `warper.warping(x, y)` 得到目标位置 $(\text{new_x}, \text{new_y})$ 。
4. 若目标位置在图像范围内，将原图颜色复制到新图像的对应位置（正向映射）。
5. 将新图像替换原图显示。

3 实验结果与分析

3.1 测试图像与控制点

采用黑白方格图像（`test.png`）作为测试图，以便清晰观察变形带来的几何扭曲。控制点设置如图 1 所示：红色线段表示从起点（蓝色圆点）到终点（绿色圆点）的位移。

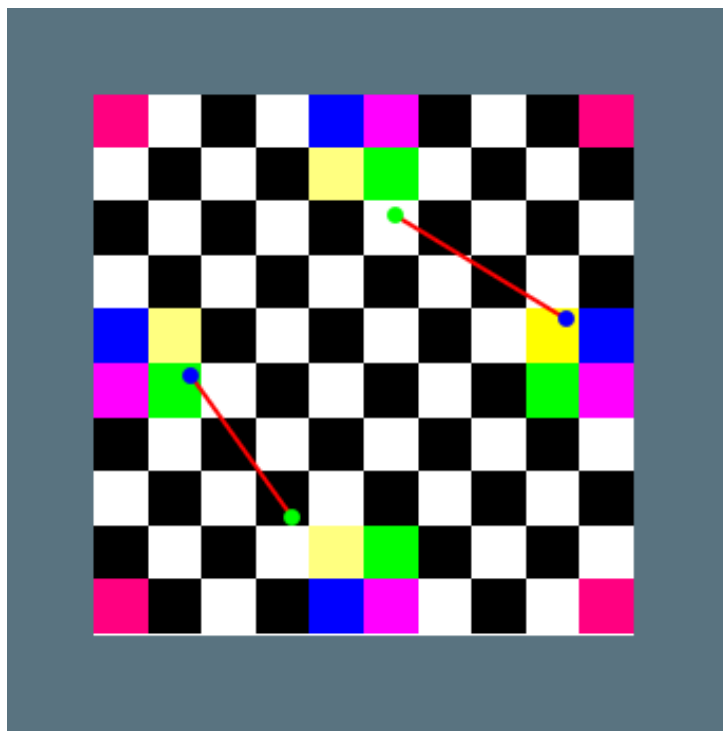


图 1: 控制点对示例

3.2 IDW 变形效果

参数 $\mu = 2$ 下的 IDW 变形结果如图 2 所示。可以看到，控制点附近的区域被拉伸或压缩，而远离控制点的区域受影响较小。整体变形连续光滑，但局部可能出现过度扭曲。

3.3 RBF 变形效果

采用多二次曲面基函数 $\phi(r) = \sqrt{r^2 + c^2}$ ，取 $c = 10$ （由控制点间距自适应计算）。结果如图 3 所示。RBF 能产生全局影响的变形，控制点之间的过渡更加自然，但参数选择（如基函数类型、形状参数）对结果影响较大。

3.4 正向映射产生的空洞问题

由于采用正向映射，多个原图点可能映射到同一目标点，而有些目标点未被任何原图点覆盖，导致结果图像中出现白色空洞（如图 4 所示）。这是正向映射的固有缺陷，需要后续处理填补。

4 讨论与改进

1. **空洞填补**：可通过反向映射（遍历目标图像每个像素，逆向查询原图颜色）避免空洞。但反向映射需要求解逆函数，通常难以解析获得，可通过数值方法或利用已

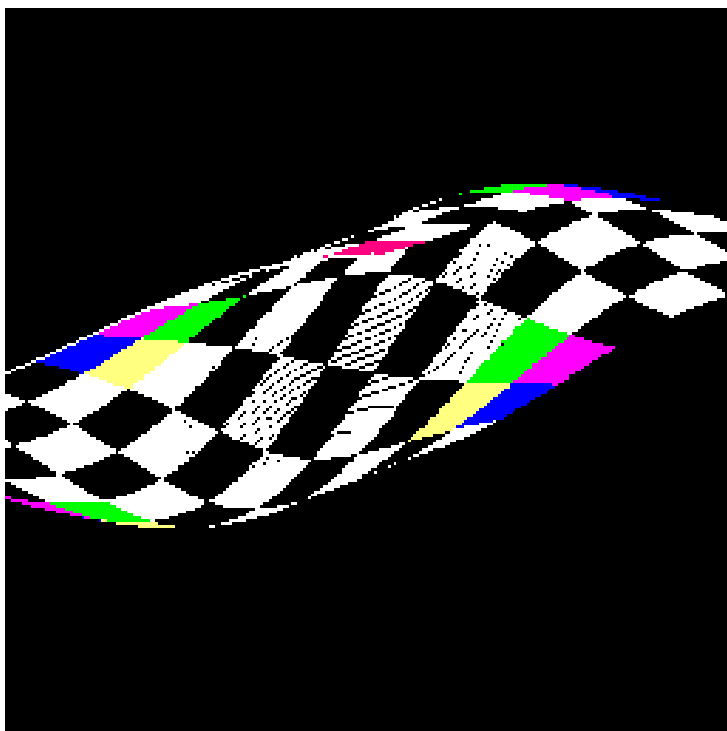


图 2: IDW 变形效果 ($\mu = 2$)

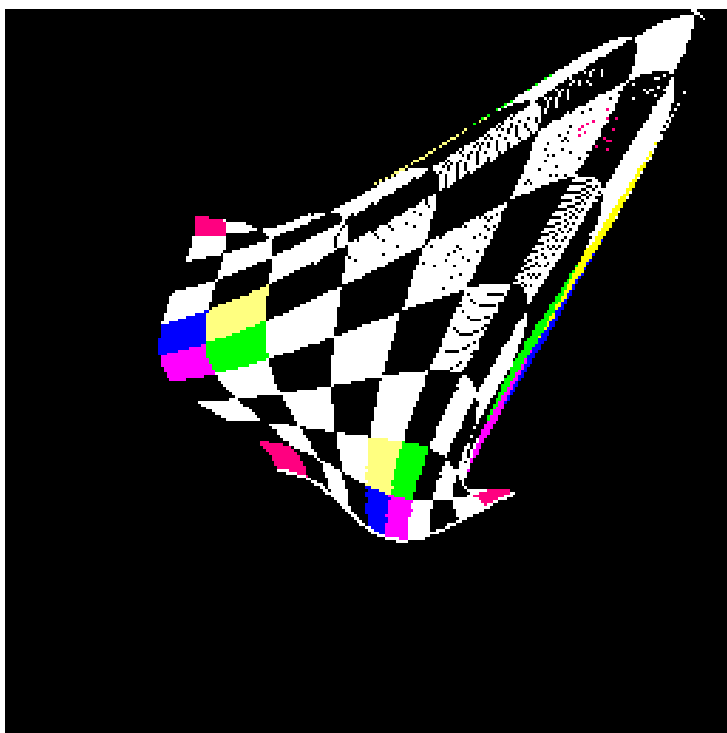


图 3: RBF 变形效果 ($c = 10$)

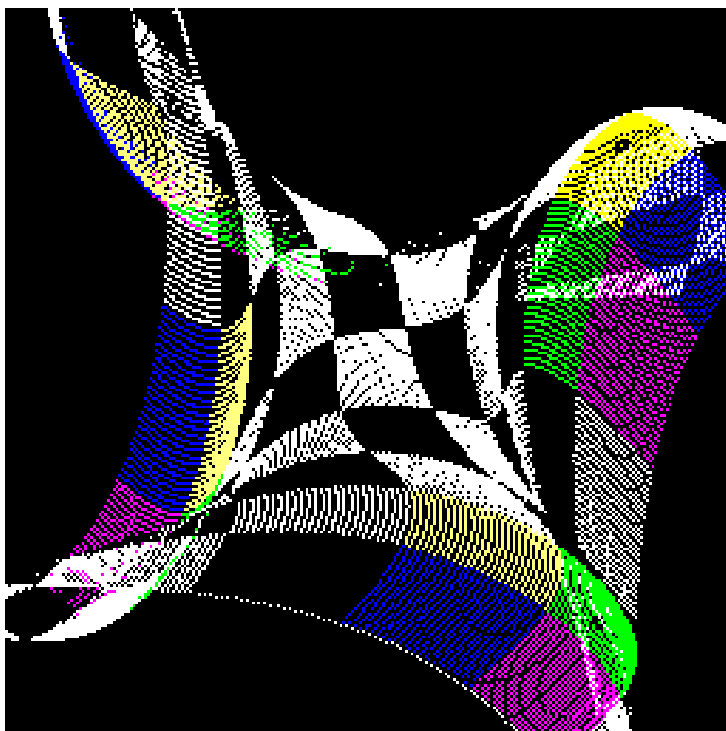


图 4: 正向映射造成的空洞

有插值函数近似。

2. **防折叠处理:** 当变形剧烈时可能出现局部折叠 (Jacobian 变负), 导致图像重叠。可采用多步缩放法: 将总变形分解为多个小步, 每一步保证 Jacobian 为正 (如 PDF 报告中所述方法)。当前实现未处理折叠。
3. **计算效率:** IDW 和 RBF 的 warping 调用涉及对每个像素遍历所有控制点, 复杂度 $O(n \times \text{pixels})$ 。可考虑使用空间划分加速邻近搜索。
4. **参数自适应:** RBF 中的形状参数 c 对结果影响显著, 可根据控制点分布自动选取 (例如取平均距离)。

5 结论

本实验成功实现了基于 IDW 和 RBF 的图像变形算法, 并通过面向对象设计将插值计算与图像处理分离。两种方法均能根据用户指定的控制点对产生合理的扭曲效果。IDW 具有局部影响特性, 计算简单; RBF 可产生全局光滑的变形, 但需解线性系统。正向映射策略带来了空洞问题, 后续可引入反向映射或插值填补来改善图像质量。

参考文献

- [1] Shepard, D. (1968). A two-dimensional interpolation function for irregularly-spaced data.
- [2] Hardy, R. L. (1971). Multiquadric equations of topography and other irregular surfaces.
- [3] Arad, N., & Reisfeld, D. (1995). Image warping using few anchor points and radial functions.